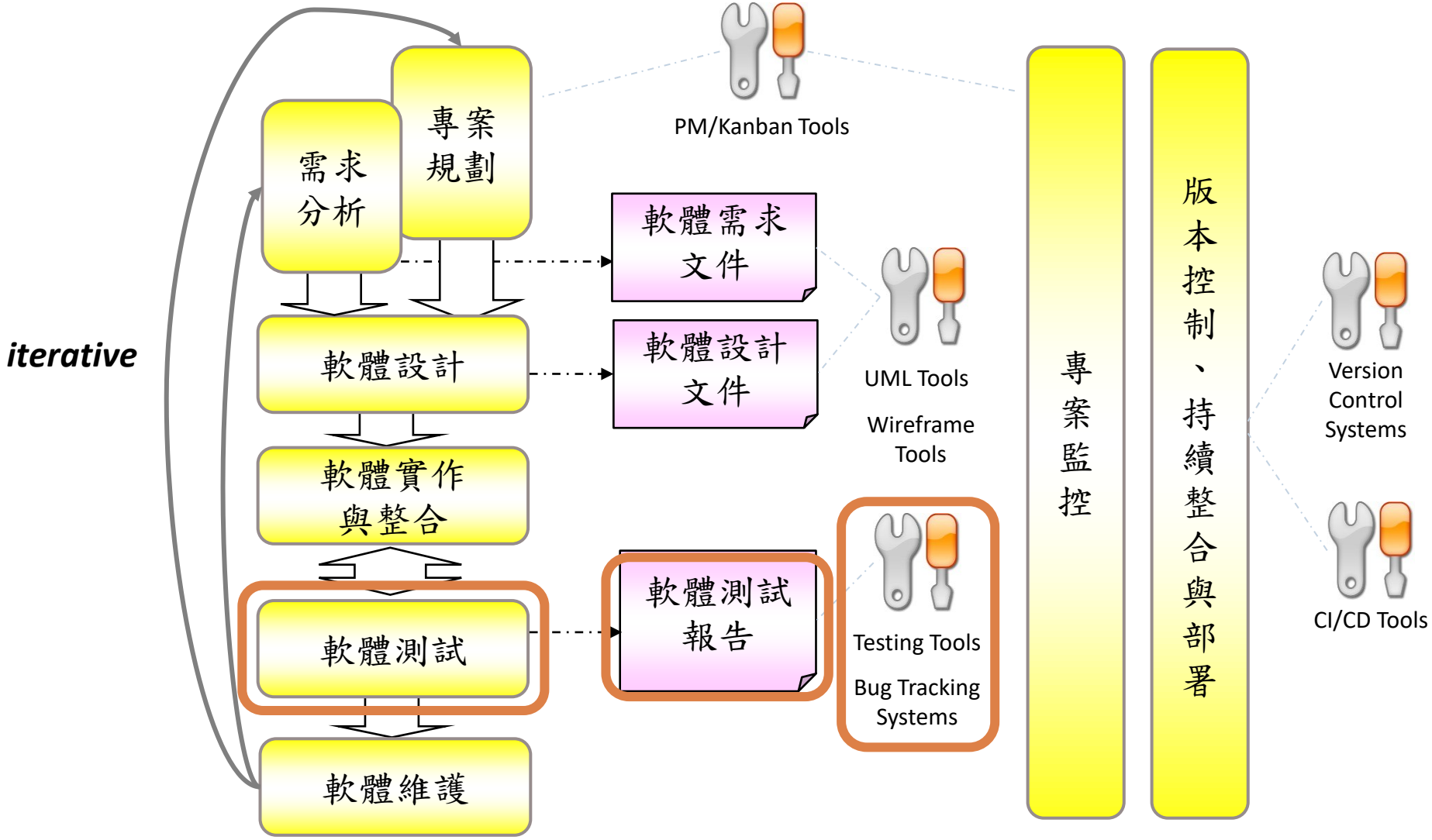
軟體工程課程:
軟體測試
(Software Testing)

海大資工 馬尚彬

大綱

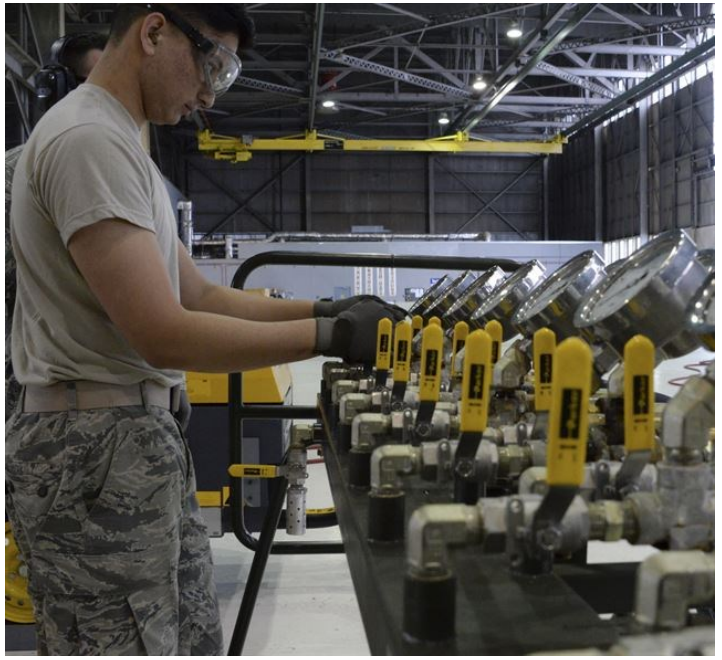
- 軟體測試概念
- 黑箱與白箱測試
- 軟體測試V模型
- 軟體測試流程
- 自動化測試

軟體開發步驟、文件與工具



軟體測試(Software Testing)

- 軟體測試是透過執行/分析待測試的程式，用以：
  - 發現程式的錯誤(bug)
  - 評估程式的品質(quality)
    - 與需求或規格一致(conformance to requirements)
    - 符合使用的目的(fitness for purpose)
    - 功能、效能、安全性、使用者體驗
  - 建立程式符合使用者需求的信心



歷史上知名的軟體錯誤

- 歷史上因軟體錯誤造成的災難或問題的案例眾多
  - 2007年高鐵售票系統全面當機
    - http://www.ithome.com.tw/node/41718
  - 2012年Knight Capital Group交易系統事故
    - https://en.wikipedia.org/wiki/Knight_Capital_Group
    - 損失約4.4億美元，公司最終被迫出售
  - 2014年eTag亂扣款
    - https://tw.news.yahoo.com/etag亂扣款-遠通被罵到爆-213000495.html
  - 2021年Facebook/Meta全球服務中斷事件
    - https://en.wikipedia.org/wiki/2021_Facebook_outage
    - Facebook市值蒸發約473億美元

測試和除錯有什麼不同?

- 在寫程式的過程中往往需要除錯，以寫出正確的程式
  - 但有做除錯的動作，不代表有進行系統化的測試，因為測試和除錯是不同的兩件工作。
- 測試和除錯的不同在於
  - 測試是發掘程式錯誤，並能有系統地展示錯誤。
  - 除錯則是找到錯誤根源，並進行程式的修正。
  - 通常測試需要執行一些測試案例，當發現程式有錯誤後，則會進行除錯，將測試所發現的錯誤排除。

不同角色的測試工作

- 開發人員
  - 測試是證明程式是對的
  - 以交付軟體驅動測試的進行(delivery-driven)
  - 主要負責單元測試、整合測試
- 測試人員
  - 測試是證明程式是錯的
  - 以驗證程式品質驅動測試的進行(quality-driven)
  - 主要負責系統測試
- 系統使用者
  - 以驗證程式是否合乎需求驅動測試的進行(requirement-driven)
  - 決定程式是否滿足驗收準則(acceptance criteria)
  - 主要負責驗收測試

軟體動態測試

- 程式撰寫完成後才能進行動態測試，執行軟體程式碼。
- 動態測試計畫需規劃測試案例(Test Case)的設計，測試的執行以及結果資料的收集和評估。
- 一般使用黑箱(Black Box)測試與白箱(White Box)測試這兩種技術來設計測試案例。
- 軟體開發過程中，透過單元測試(Unit Testing)、整合測試(Integration Testing)、系統測試(System Testing)與驗收測試(Acceptance Testing)等各種軟體測試階段，來檢查軟體介面、功能、與非功能需求。

軟體動態測試的問題


```
int scale(int m, int n, int p) {
    double j=0.0 ;
    for (i=0 ; i<n ; i++) {
        if (m>n) {
            j=j-1;    // 正確應為j=j+1;
            j=j/p;
        }
    }
    return j;
}
```

如何針對上述程式碼片段進行完整的軟體動態測試?

在電腦中int為32 bit (4 bytes)，若要完全測試所有m, n, p的輸入組合，則必須測試 $2^{32} \times 2^{32} \times 2^{32}$ 組= $2^{96} \approx 7.92 \times 10^{28}$ 組。

即使以極度樂觀的速度(每秒 10^{18}組)去進行這些測試資料組的測試，仍要約 2,500 年。

測試與診療類比

- 想像一個病人去醫院看診，醫師怎麼檢查病徵與病因？
  - 看外觀與反應(印堂發黑、眼白泛黃)、敲打測試是否疼痛(右腹敲打是否會痛？)
  - 超音波、胃鏡、腸鏡
- 測試跟診療非常類似！
  - 黑箱測試與白箱測試跟上面的例子是否很像？
  - 看診很快是好是壞？
  - 看診很慢是否就可以找到病因？

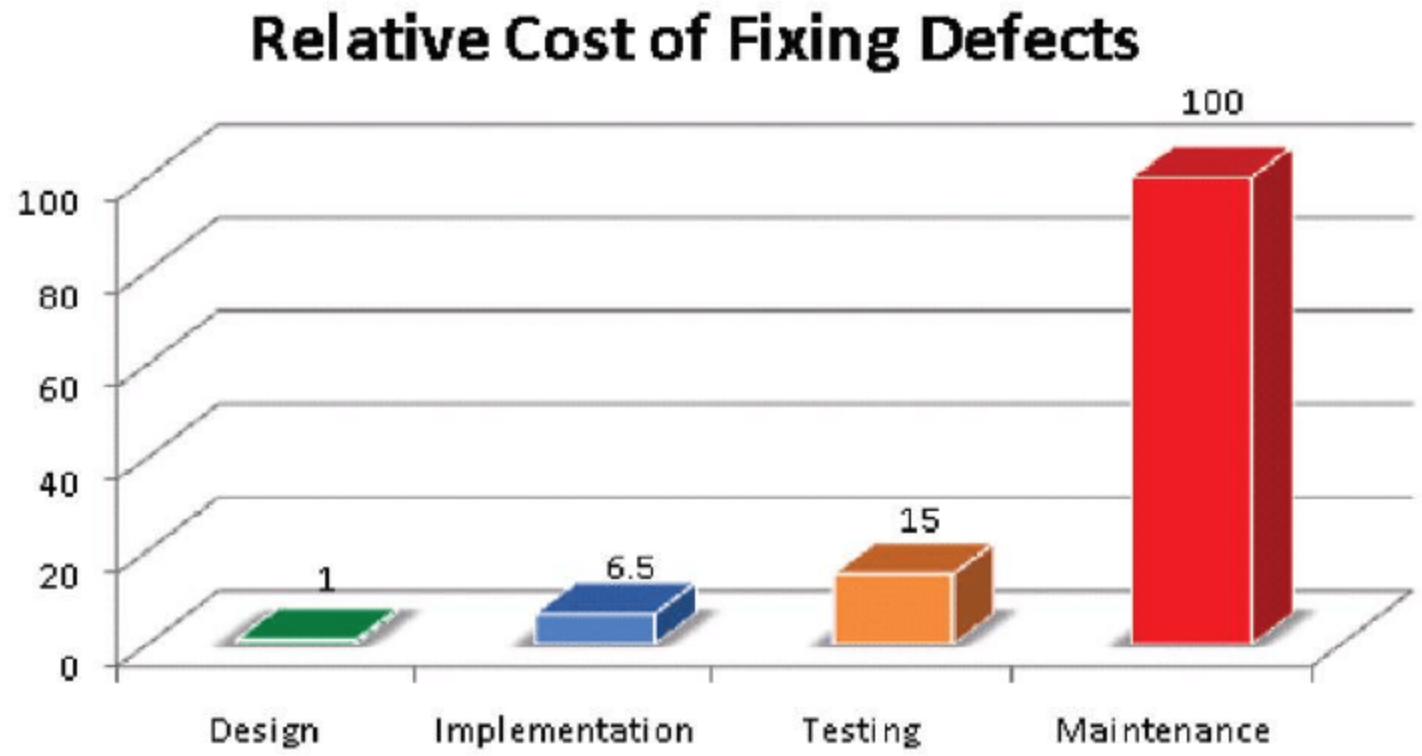
窮舉性測試(Exhaustive Testing)

- 窮舉性測試(Exhaustive Testing)
  - 測試所有的輸入組合與路徑
  - 實務上不可行(impractical)
- 選擇性測試(Selective Testing)
  - 選擇測試一些輸入組合與路徑，有效地偵測出程式的錯誤
  - 如何選擇，以及選擇多少測試案例才足夠?
    - 可藉由測試方法、測試覆蓋率和經驗的協助
  - 隨意或盲目的選擇，有可能耗費寶貴的測試資源(時間和成本)，卻無法達到測試目的

軟體動態測試的主要目的

- 為了能夠在有限的時間內，以最有效率的方式發現最多的軟體錯誤，而發現1個或多個尚未發現的錯誤，都可算是一次成功的測試結果。
- 測試案例設計的好壞決定整個動態測試程序的成效。
  - 越早發現錯誤(缺失)，修正的成本越低。
- 測試中只發現少量的錯誤，其可能表示的意義有以下兩種可能：
  - 1. 表示此軟體系統的品質已相當可靠及穩定。
  - 2. 測試工作準備不夠充分或執行不夠確實，這是產品將來可能產生危機的警訊。

缺失排除的相對代價



https://www.researchgate.net/publication/255965523_Integrating_Software_Assurance_into_the_Software_Development_Life_Cycle_SDLC/figures?lo=1&utm_source=google&utm_medium=organic

動態測試案例設計技術

動態測試案例設計1

- 白箱測試(White-box testing)
  - 了解軟體內部結構後進行測試。
    - 根據程式碼判斷條件或迴圈，定義邏輯路徑，產生測試案例資料。
  - 有效測試程式細節，確保程式的穩定性。
    - 對程式結構或資料變數使用的錯誤特別有效。
  - 缺點
    - 成本高。
    - 對需求規格錯誤的效果不好。
    - 對介面錯誤的效果不好。

動態測試案例設計2

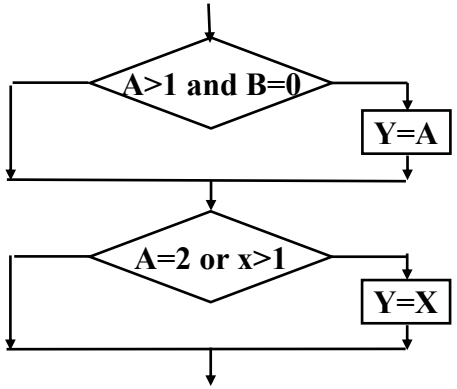
- 黑箱測試(Black-box testing)
  - 了解軟體產品需求功能後進行測試。
  - 不考慮軟體內部邏輯的結構。
  - 測試軟體介面，正確輸入、輸出。

測試覆蓋準則(Coverage Criteria)

- 敘述覆蓋(Statement Coverage)
  - 測試案例要能使程式每一敘述至少執行一次
- 分支決策覆蓋(Branch Coverage)
  - 測試案例要使程式每一決策點至少執行一次 (True/False各一次)
- 條件覆蓋(Condition Coverage)
  - 所有邏輯判斷情況都至少執行過一次 (True or False)
- 決策／條件覆蓋(Branch/Condition Coverage)
  - 結合分支決策覆蓋與條件覆蓋
- 多重條件組合覆蓋(Multiple Condition Coverage)
  - 不同組合的判斷情況都至少被執行一次(同一行條件true/false之排列組合)
- 全面路徑覆蓋(All-Paths Coverage)
  - 測試案例要使軟體每一路徑至少執行一次(跨行條件true/false之排列組合)

Statement Coverage

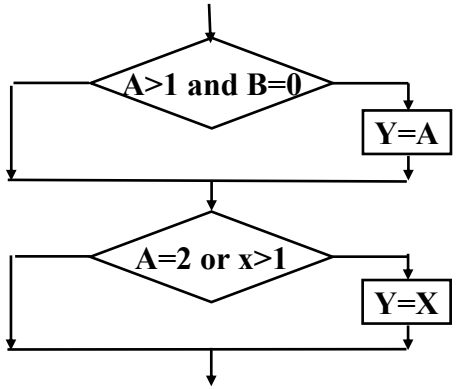

```
10 INPUT A,B,X
20 IF (A>1) AND (B=0) THEN Y=A
30 IF (A=2) OR (X>1) THEN Y=X
40 PRINT Y
```



- 設計測試案例，使每一條指令敘述至少執行一次
  - input為(2,0,3)就可覆蓋所有可執行指令，結果會得到3
- 敘述覆蓋測試方法較不嚴謹，例如若把：
  - 行號20的AND改成OR
  - 行號30的X>1改成X>0
  - 行號20後面的Y=A改成其他的敘述
- 程式執行結果沒變，所以找不出這種錯誤。
- 敘述覆蓋最弱邏輯涵蓋準則，白箱測試至少要做到此測試。

Branch (Decision) Coverage

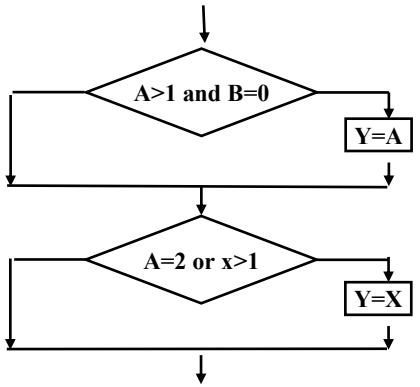

```
10 INPUT A,B,X
20 IF (A>1) AND (B=0) THEN Y=A
30 IF (A=2) OR (X>1) THEN Y=X
40 PRINT Y
```



- 分支決策覆蓋目標是設計測試案例，使程式每個判斷取真分支和取假分支至少執行一次。
  - A,B,X若為(3,0,3)和(3,1,1)能使得 (A>1) AND (B=0)和(A=2) OR (X>1)這兩個布林運算式均能產生真和假的值。
  - 若將 (A>1)寫成(A>2)卻是無法測出的錯誤。
  - 決策涵蓋測試方法嚴密性比敘述涵蓋高。

Condition Coverage

- 條件覆蓋和分支覆蓋類似，不過條件覆蓋是以某項條件為主，而決策覆蓋則以整個布林運算式為主。使程式中**每個判斷的每個條件至少執行一次**。
- 案例說明：
  - 前述程式具有下列四條件 A>1、B=0、A=2、X>1
  - 欲使這四個條件都能產生真與假的值，測試案例須包含
    - (1) A>1, (2) A<=1
    - (3) B=0, (4) B!=0
    - (5) A=2, (6) A!=2
    - (7) X>1, (8) X<=1
  - (1,0,3)滿足2、3、6、7
  - (2,1,1)滿足1、4、5、8
  - 但這兩組測試案例並未使運算式(A>1) AND (B=0)和(A=2) OR (X>1)具有真值與假值 ，亦即未達成100% Branch Coverage。



Branch Coverage and Condition Coverage

- 條件覆蓋嚴密性似乎比分支決策覆蓋高，但並非絕對。
  - 如上頁案例即未達成100%分支決策覆蓋。

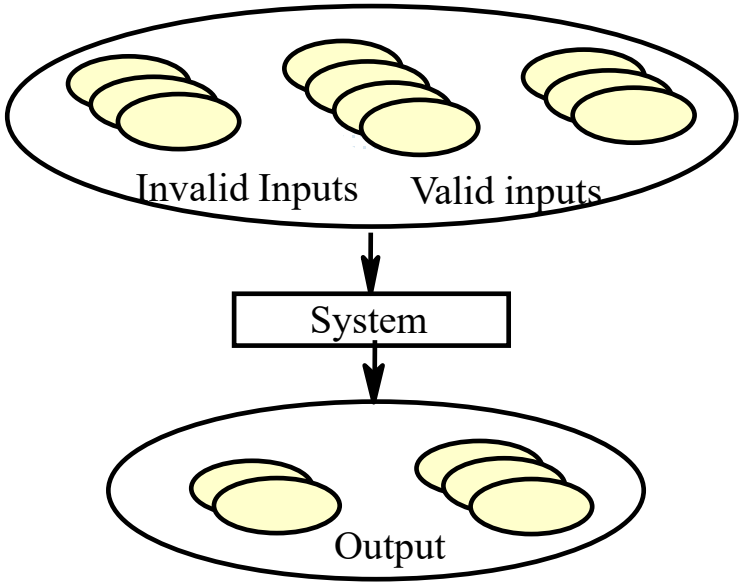
- 決策/條件覆蓋(Branch/Condition Coverage) ：需設計足夠的測試案例，使判斷敘述的布林運算式中的每個條件的所有可能值都至少執行一次，同時每個判斷敘述的所有可能判斷結果都至少執行一次。
  - 例如，輸入值(2,0,3)和(1,1,1)即可達成此目標。

黑箱等價劃分

Equivalence Partition

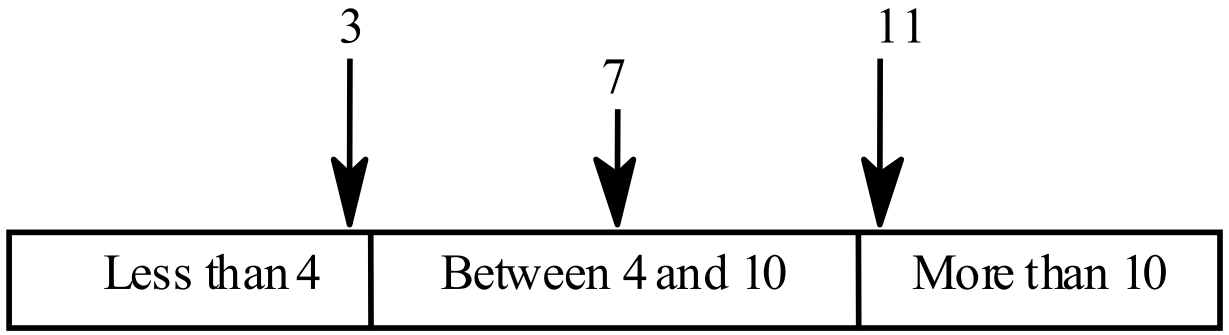
黑箱等價劃分簡介

- 以不同資料類別、劃分規則去切割輸入域與設計測試案例。
  - 訂出**輸入域的子集合**：各輸入資料對程式中的錯誤都是等效的。
  - 同一類測試案例，會發現某類同樣錯誤。



等價劃分範例1

- 輸入條件規定值的範圍a和b ，則取一個有效等價類別a<x<b，和兩個無效等價類別<a、>b的值設計測試案例
  - 輸入 x 為 ，4 < x < 10
  - 選擇測試案例為邊界值：3, 7, 11
  - 選擇測試案例為：負數、非數字



等價劃分範例2

- 輸入資料須遵守某規則
  - 一個有效等價類別和若干個無效等價類別
- 範例：輸入條件為 非0非1開始的3位數字
  - 有效等價類：200–999 (首位非 0/1 的三位數，如300)
  - 無效等價類：以 0 開頭 (如 011)、以 1 開頭 (如 100)、非三位數等。

等價劃分範例3

- 假設我們開發一個程式根據學生的數字分數(numerical score)來決定成績的等級(letter grade)。該程式的輸入變數是Score，介於0和100之間，成績等級的計算方式如下：

Score	Grade
90~100	A
80~89	B
70~79	C
60~69	D
0~59	F

等價劃分範例4

- 我們可以分割Score的定義域為1個有效輸入與2個無效輸入的等價類別
  - 1個有效輸入等價類別是score介於0和100之間
  - 2個無效輸入等價類別分別為score小於0和score大於100
  - 此外，依據成績等級計算的規則，我們又可將有效輸入等價類別分割成5種不同的等價分類：0~59、60~69、70~79、80~89和90~100，分別代表不同的輸出成績：A、B、C、D和F
- 對測試而言，在每個等價類別內的輸入資料，會有相同的結果。因此，只需挑選一個輸入資料做為代表
  - 以這個程式範例，我們可以將有效區間的測試案例從101個，即測試score所有可能的值(假設0<=score<=100)，降低至5個。

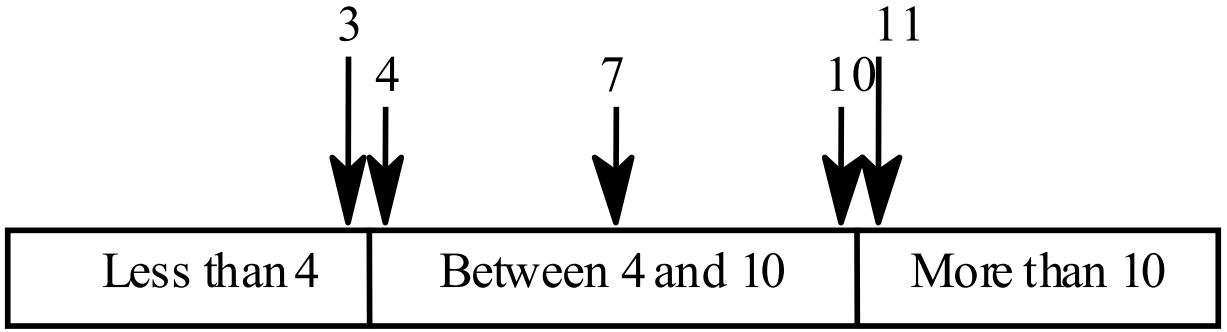
黑箱邊界值分析
Boundary Value Analysis

邊界值分析

- 邊界值分析(Boundary Value Analysis, BVA)
  - 為等價劃分的擴充
  - 邊界值通常是等價類別的界限
  - 錯誤發生在輸入域邊界上的機率遠大於輸入域的中間
    - BVA選擇等價類別為“邊界”值
    - BVA也可根據輸出域設計測試案例

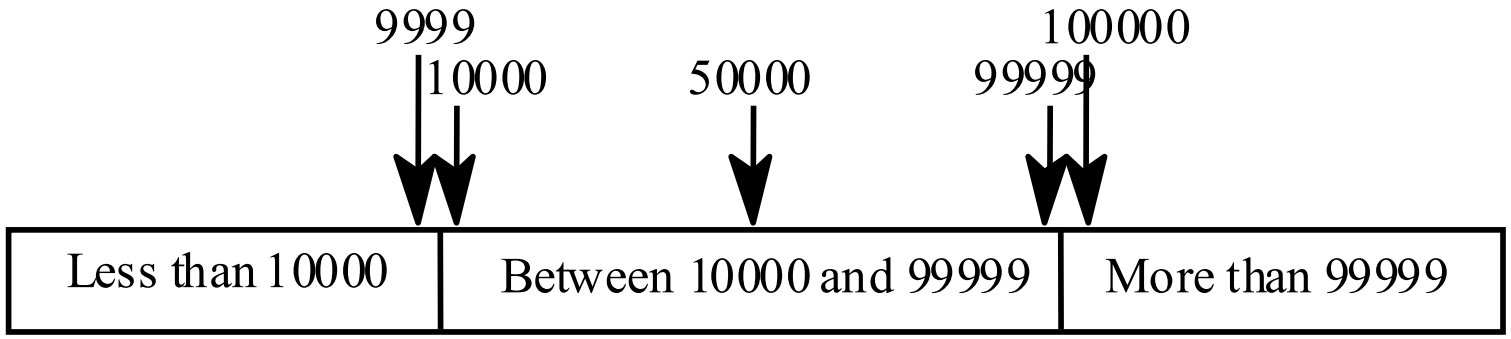
邊界值分析範例1

- 輸入條件規定值的範圍a和b ，則取剛好小於、等於a，和剛好大於、等於b的值設計測試案例
  - 輸入 x 為 4 <= x <= 10
  - 選擇測試案例為邊界值：3, 4, 10, 11
  - 再增加一般值7



邊界值分析範例2

- 輸入條件規定一個數值，如整數，則取剛好大於最小值及剛好小於最大值
  - 輸入 x 為 5位整數，
  - 10000<=x<=99999 ，選擇測試案例為邊界值
    - 9999, 10000, 99999, 100000
  - 再增加一般值50000

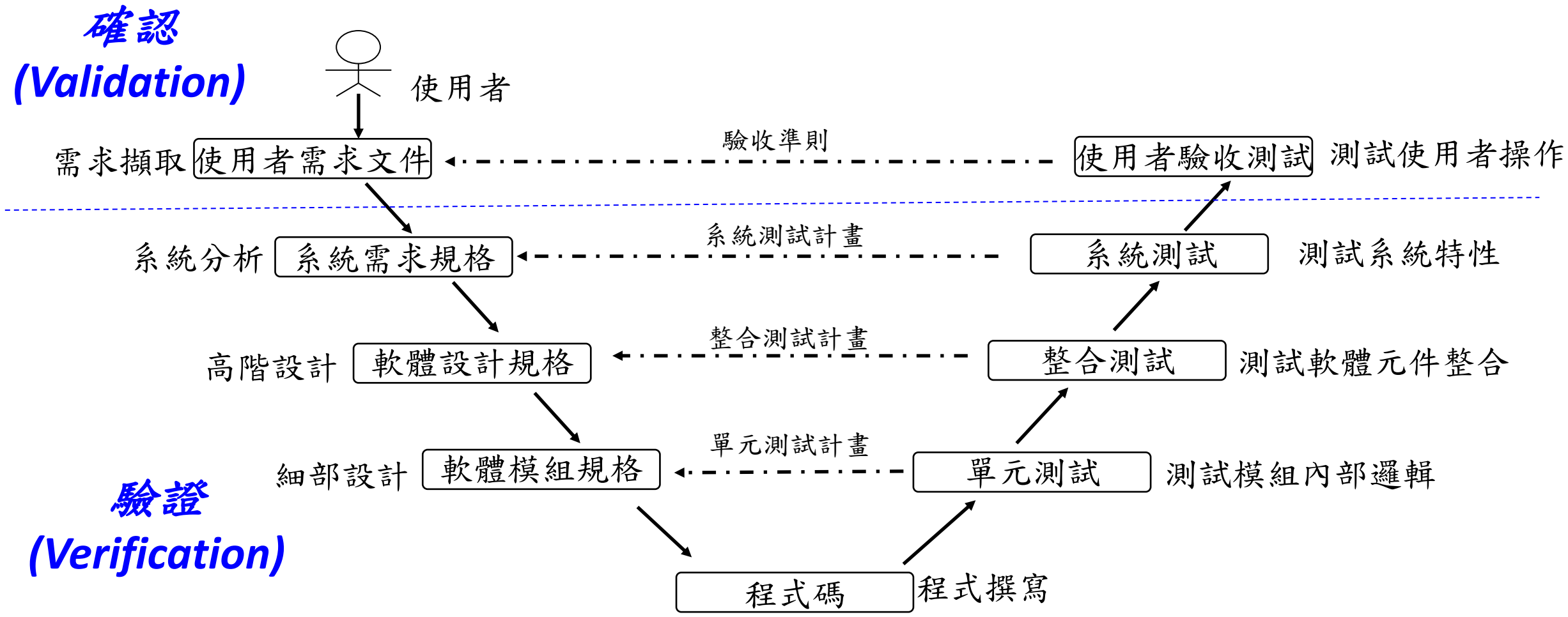


邊界值分析範例3

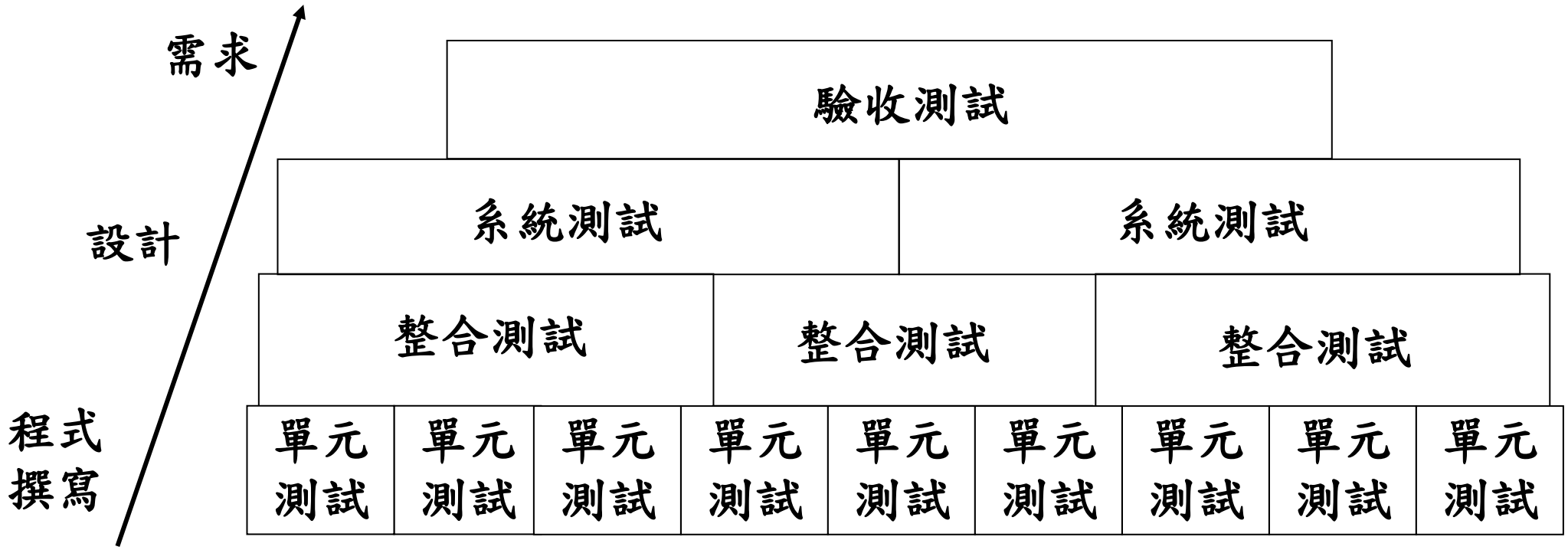
- 根據規格，每個輸出條件，可使用上述原則。
  - 輸出溫度壓力表。
  - 產生最大(或最小)可允許壓力、溫度值。
  - 也可考慮一般值。

動態測試策略

軟體測試 V 模型


確認
(Validation)
使用者
需求擷取
使用者需求文件
驗收準則
使用者驗收測試
測試使用者操作
系統分析
系統需求規格
系統測試計畫
系統測試
測試系統特性
高階設計
軟體設計規格
整合測試計畫
整合測試
測試軟體元件整合
細部設計
軟體模組規格
單元測試計畫
單元測試
測試模組內部邏輯
驗證
(Verification)
程式碼
程式撰寫

軟體動態測試程序

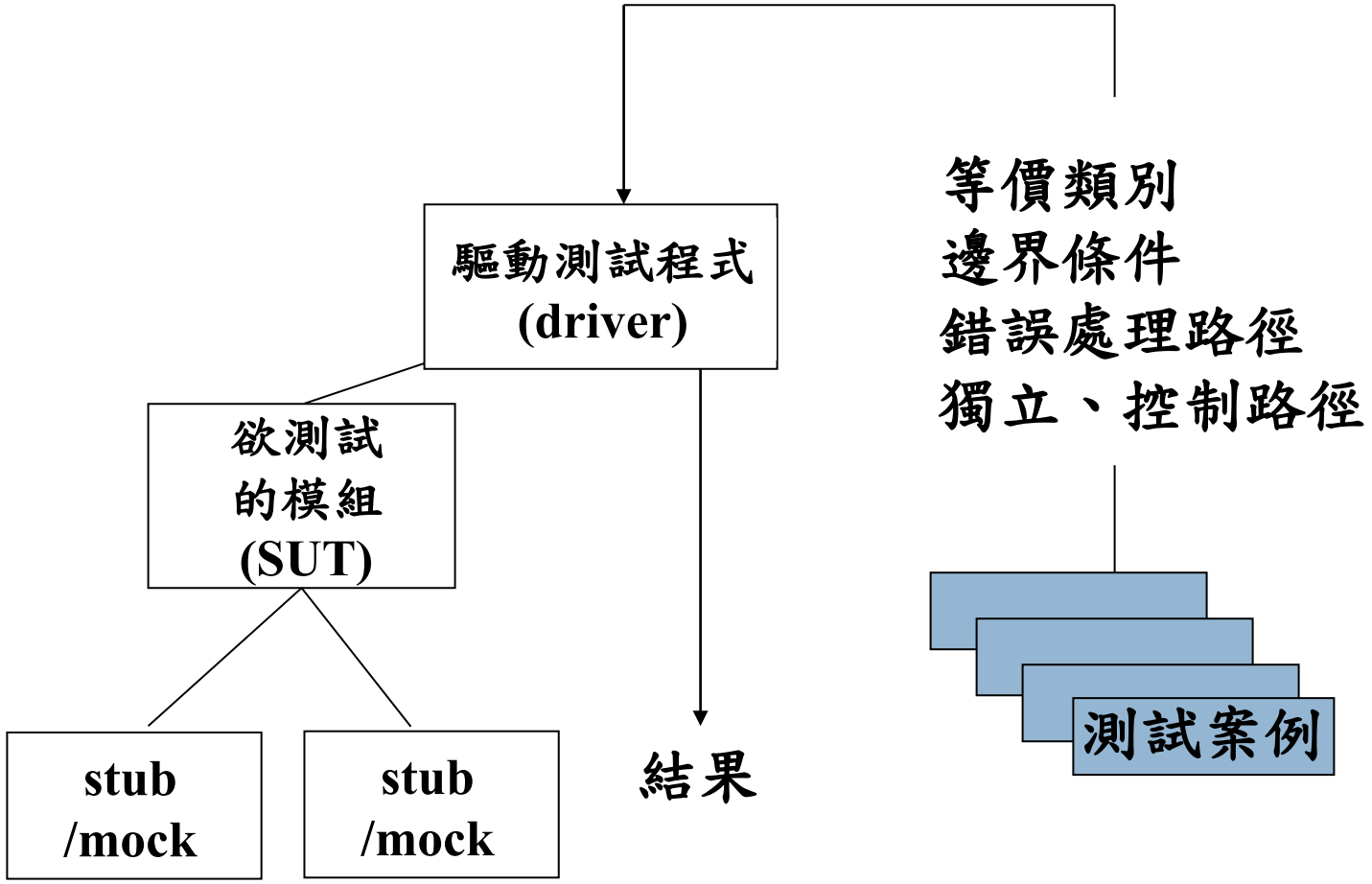


測試方向與工作產物

單元測試

- 目標：找出單元模組內部功能、邏輯錯誤。
- 方法
  - 靜態分析
  - 動態測試：可同時對多個模組平行測試
- 人員：由軟體開發人員測試
- 執行重點：注意模組對錯誤的控制能力
- 單元設計原則
  - 遵循高內聚力設計模組，簡化單元測試。
  - 當一個模組僅實現一個功能時，可降低測試案例數量，且容易發現錯誤。
- 重構(Refactoring)建立在通過單元測試的前提下。

單元測試架構


驅動測試程式
(driver)
欲測試
的模組
(SUT)
stub
/mock
stub
/mock
結果
等價類別
邊界條件
錯誤處理路徑
獨立、控制路徑
測試案例

Test Driver與Test Double

- SUT (System Under Test或Software Under Test)：代表待測程式。
- DOC (Depended-on Component)：相依元件，是SUT執行的時候會使用到的元件。
  - 要測試SUT，DOC也必須存在。
  - 例如：Engine物件是Car物件的DOC。
- 驅動測試程式(Test Driver)：可以視為「主程式」，接收測試案例的資料，傳遞到被測試模組，再比較預期結果。
- 測試替身(Test Double)：讓SUT可以不依靠DOC而可單獨被測試的作法，提供“替身”讓SUT使用。
  - Dummy, Stub, Spy, Fake, Mock

Test Doubles[1]

Test Double Type	Description
Dummy	The simplest, most primitive type of test double. Dummies contain no implementation and are mostly used when required as parameter values, but not otherwise utilized. Nulls can be considered dummies, but real dummies are derivations of interfaces or base classes without any implementation at all.
Stub	A step up from dummies, stubs are minimal implementations of interfaces or base classes. Methods returning void will typically contain no implementation at all, while methods returning values will typically return hard-coded values.
Spy	A test spy is similar to a stub, but besides giving clients an instance on which to invoke members, a spy will also record which members were invoked so that unit tests can verify that members were invoked as expected.
Fake	A fake contains more complex implementations, typically handling interactions between different members of the type it's inheriting. While not a complete production implementation, a fake may resemble a production implementation, albeit with some shortcuts.
Mock	A mock is dynamically created by a mock library (the others are typically produced by a test developer using code). The test developer never sees the actual code implementing the interface or base class, but can configure the mock to provide return values, expect particular members to be invoked, and so on. Depending on its configuration, a mock can behave like a dummy, a stub, or a spy.

https://docs.microsoft.com/zh-tw/archive/msdn-magazine/2007/september/unit-testing-exploring-the-continuum-of-test-doubles

Test Doubles2

- Dummy
  - 只為了「佔位置」的參數，**不會被用到**
  - 直接傳null，或new一個隨便的物件
- Stub
  - 簡單「**固定回傳**」的替身
  - 隔離外部相依、回傳預期資料
- Spy
  - 如同 Stub，但會**記錄呼叫**，可驗證被怎麼用
  - 需要知道「有沒有被呼叫、用什麼參數」

Test Doubles3

- Fake
  - 有可運作的簡化實作(例如運用Memory DB)
  - 快速完成整合/邏輯測試
- Mock
  - 由框架動態產生(例如Mockito)，著重互動驗證
  - 關注「有沒有照規格呼叫協作者」

單元測試: JUnit 5


```
//將要被測試的類別
public class Car {
    public int getWheels() {
        return 4;
    }
}
```

```
//測試案例(Partial)
@BeforeEach
public void setUp () throws Exception {
    System.out.println("Before");
    expectedWheels = 4;
    myCar = new Car();
}
@AfterEach
public void tearDown () throws Exception {
    System.out.println("After");
    myCar = null;
}
@Test
public void testGetWheels () {
    assertEquals(expectedWheels, myCar.getWheels());
}
```

You can use Jest to test JavaScript:
https://jestjs.io/

JUnit常用Annotation

- @Test: 測試方法
- @Disabled: 忽略的測試方法，設定為暫時不執行
- @BeforeEach (JUnit4: @Before): 每一個測試方法之前執行，有多少個測試方法就執行多少次
- @AfterEach (JUnit4: @After): 每一個測試方法之後執行，有多少個測試方法就執行多少次
- @BeforeAll (JUnit4: @BeforeClass): 所有測試開始之前執行 (預設為static)
- @AfterAll (JUnit4: @AfterClass): 所有測試結束之後執行 (預設為static)
- @DisplayName: 自訂測試名稱(讓報告更清楚)

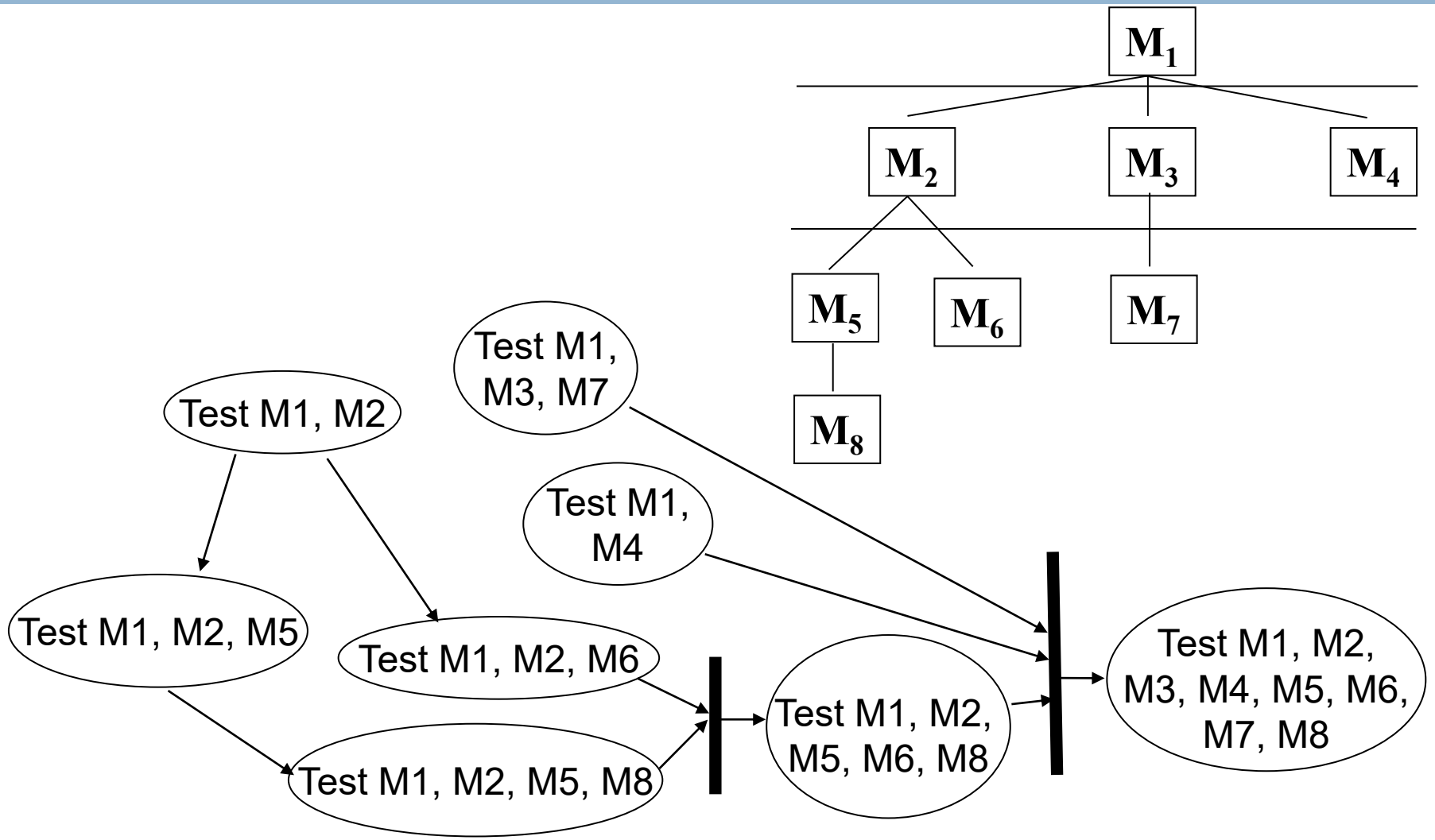
JUnit常用API

- assertEquals 判斷2個物件是否相等
- assertNotEquals 判斷2個物件是否不為相等
- assertTrue 驗證是否為真
- assertFalse 驗證是否為偽
- assertNull 驗證物件是否為null
- assertNotNull 驗證物件是否不為null
- assertArrayEquals 驗證陣列是否相同
- assertThrows 例外驗證
- assertTimeout 超時驗證

整合測試

- 測試時機
  - 模組元件通過單元測試。
  - 若部份單元完成單元測試，其餘尚未發展完畢，可先整合已完成單元測試的單元，進行整合測試。
- 參與人員
  - 開發人員協助進行整合測試。
  - 當軟體整合結構完成，測試小組便可獨立工作。
- 測試目的
  - 整合模組間介面一致性問題，測試系統整合功能。
  - 資料經過模組介面，可能被忽略。
  - 一個模組可能對其它模組產生無意的不良影響。
  - 個別模組可接受的不精確資料，經過其他模組，可能被放大到不可接受的地步。
  - 全域性資料可能被其他模組無意修改，造成整合問題。

由上而下的整合


M1
M2
M3
M4
M5
M6
M7
M8
Test M1, M2
Test M1, M3, M7
Test M1, M4
Test M1, M2, M5
Test M1, M2, M6
Test M1, M2, M5, M8
Test M1, M2, M5, M6, M8
Test M1, M2, M3, M4, M5, M6, M7, M8

由下而上的整合

- 整合原則
  - 從子模組(程式結構中最低層模組)開始進行整合測試
  - 模組由下而上整合，不需stub模組
- 整合程序
  - 合併低層模組成一組模組，執行特定系統子功能
  - 撰寫驅動模組(進行測試的控制程式)，協調測試案例的輸入和輸出。
  - 測試每一組模組。
  - 刪除驅動模組，在程式結構中向上合併模組群

系統測試

- 測試系統軟體與硬體整體功能是否協調。測試環境應與實際環境相似。以黑箱測試技術為主。
- 由硬體工程師與軟體工程人員進行系統測試。
  - 系統測試：必要時可由系統分析人員兼任
  - 系統整合測試：具良好溝通能力及完整概念的資深測試人員
- 系統測試
  - 測試目的：確認開發人員依照系統需求文件正確無誤開發系統。
  - 測試方法：驗證系統需求文件所描述功能都正確無誤實作，以驗證系統內部功能。可使用模擬器。
  - 注意事項：若自己實作的模組由自己測試，可能因盲點而找不到問題。

驗收測試

- α測試
  - 軟體由開發人員指導使用者使用，同時記錄錯誤和使用中的問題。
- β測試
  - 由軟體的最終使用者測試，開發人員不參與。
  - 使用者記錄在測試期間遇到的所有問題，定期報告給開發人員修改。
- 使用者驗收測試(User Acceptance Testing, UAT)
  - 確認開發團隊所開發的系統能達成商業目的。
  - 使用者代表(資深使用者、熟悉系統及商業流程者)參與。

迴歸測試(Regression Test)

- 系統軟硬體部份變更或修改，驗證此修正不會引起不良影響，確保所有功能需求仍正常運作。
  - 系統版本更新：如除錯或功能強化更新。
  - 配合軟體升級：如3-party函式庫升級、外部介接軟體系統升級。
  - 軟體環境升級：如使用不同廠商瀏覽器或其瀏覽器升級、資料庫系統版本升級、作業系統升級等。
  - 硬體升級：如伺服器更新升級、網路基礎建設更新。
  - 新的效能需求。

效能(Performance)測試

- 所有測試階段都可實施效能測試。例如進行單元白箱測試時，也可測試單元模組的效能。
- 所有系統模組完全整合後，系統真正效能才能確定。
- 需硬體和軟體配合。
  - 以嚴格方式度量資源效用(如處理器周期)。
  - 使用外部設備監控執行間隔，記錄發生事件，如中斷情況。
- 可用軟體工具：LoadRunner、JMeter

測試流程與計畫

軟體測試步驟

- 軟體測試程序中的任一軟體測試階段，一般都可以分成以下四個主要的執行步驟：
  - (1)**測試規劃**：測試前應擬定測試目標與測試計畫
  - (2)**測試準備**：準備所有參考資料及測試工具與環境
  - (3)**執行軟體測試**：執行實際測試與記錄錯誤與缺失
  - (4)**評估與控制**：監控與檢討測試成效

測試規劃

- 測試工作必須配合計畫的開發時程，因此透過撰寫測試計畫書，才能以有系統化的方法準備與進行測試工作。
- 撰寫測試計畫書的主要目的為：
  - 讓參與的測試人員彼此間的溝通與分工更加順利，以達成測試目標。
  - 以系統化的方式進行軟體測試，便於管理。

測試準備

- **依測試計畫書進行準備**：包括測試人員的相關訓練、測試需要的軟硬體設備、文件資料的準備、測試輔助工具的準備以及教育訓練工作。
- **完成測試案例設計**：測試設計文件中應詳細說明測試步驟與測試資料，並有預期結果與實際測試結果之比較。
- **撰寫測試程式或採購測試工具**：依據測試的要求，開發額外的測試程式以便完成測試步驟；或是採購現有的測試輔助工具以進行測試。

測試案例(Test Cases)設計方法

- 一個測試工作是由許多的測試案例的執行所構成，而測試案例中將記錄所有相關的測試資訊，以輔助測試人員進行測試，找出未發現的錯誤。
- 完整的測試案例可以包含編號、名稱、目標、條件、輸入資料、測試步驟、期望結果等資訊。
- 在軟體系統發展初期就開始設計測試案例(只要有辦法收集到足夠的測試資訊)，並視需要開始進行測試，並非一定要等到所有的軟體元件都開發完畢才開始進行測試案例的設計。

測試案例1

Identification	BBDP-TC-P19
Name	新增問卷測試
Reference	BBDP-FR-P09
Severity	1
Instructions	1. 點選我的問卷，進入問卷頁面 2. 點選新增問卷 3. 選擇醫院、科別、醫師、症狀 4. 填寫所有問題 5. 點選送出
Expected result	傳送推播至該醫生端系統，醫生點擊推播後可查看該問卷。
Cleanup	無

測試案例2

- 設計測試案例的原則為
  - 能具有合理的機率偵測到軟體錯誤，以避免完成所有的測試，卻無法提升軟體品質，浪費執行成本。
  - 不重複出現相同的測試案例，以避免過多無用的測試案例，無法發現新問題，且亦浪費測試執行與案例管理的成本。
- 好的測試案例
  - 以有效的方法產生，有較高的機率偵測出錯誤。
  - 可以明顯的呈現、追蹤軟體的錯誤。
  - 針對測試需求，不會太複雜，也不會太簡單，容易執行測試。

執行軟體測試1

- 測試輸入：包括兩種組態，一種是軟體組態，例如，軟體需求規格、設計規格和原始碼等；另一種為測試組態，例如，測試計畫、工具、測試案例、測試資料和預期結果等。
- 進行測試取得測試結果，並比較測試結果與預期結果。預期結果與實際結果若存在萬分之一的差異，都可能需花費相當長時間進行診斷和修正，不可以輕易的忽略。

執行軟體測試2

- 若找到錯誤或缺失，應輸入追蹤系統，交由開發人員除錯。若已於下一版本修正問題，則測試工程師需再次使用同一測試案例測試此問題，直到確認問題已修正完畢為止。
  - 追蹤系統範例：
    - Mantis (https://www.mantisbt.org/bugs/my_view_page.php)
    - GitHub Issue Tracker (https://github.com/hubotio/hubot/issues/1494)
    - Jira (https://www.atlassian.com/software/jira)
- 收集測試結果與進行評估，可透過歷史數據來預測將來可能發生的錯誤數量。

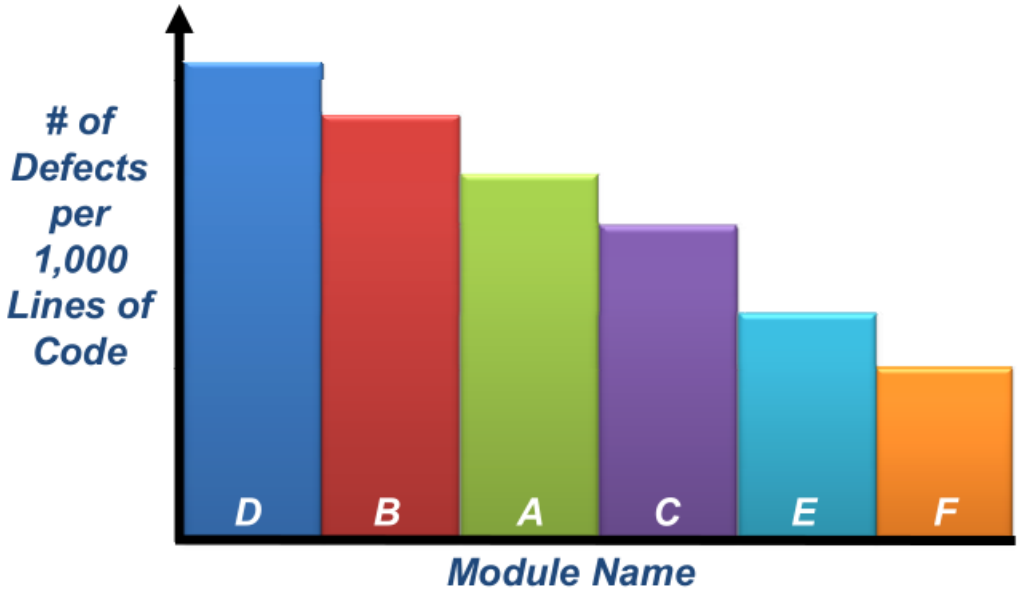
測試結果報告

測試案例編號	測試結果 (Pass/Fail)	註解
XYZ-TC-001		
XYZ-TC-002		
XYZ-TC-003		
XYZ-TC-004		
XYZ-TC-005		
XYZ-TC-006		
XYZ-TC-007		
XYZ-TC-008		
XYZ-TC-009		
XYZ-TC-010		
RATE	? %	

測試結果分析

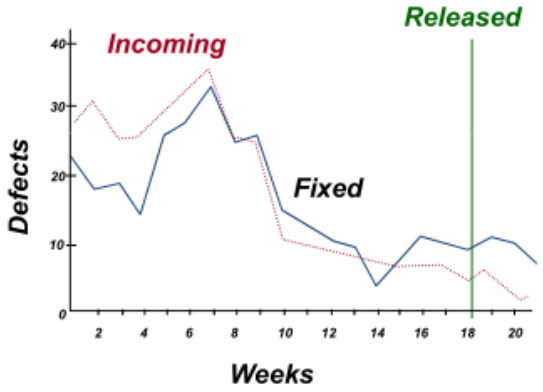
SQE TRAINING

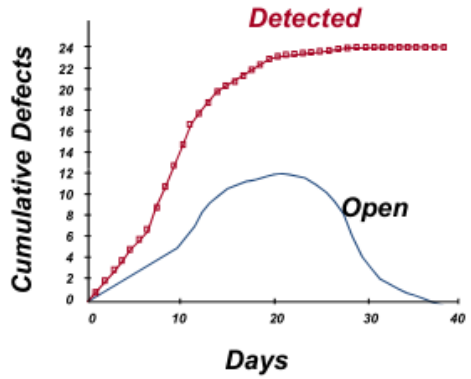
Defect Density/Clustering



SQE TRAINING

Open and Closed Over Time





http://www.slideshare.net/TechWellPresentations/measurement-and-metrics-for-test-managers-37098471

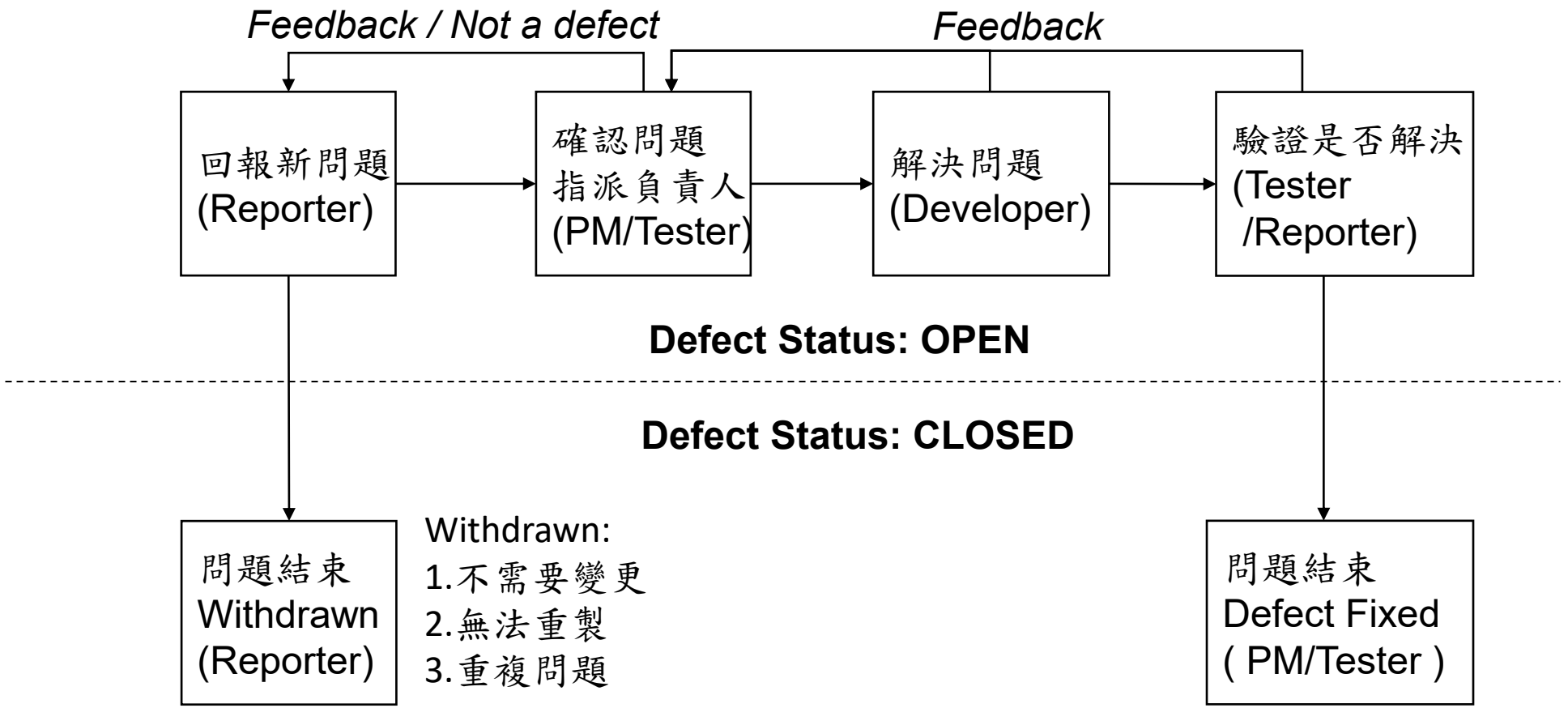
執行軟體測試標準原則

- 先測核心功能(kernel)，再測輔助功能
- 先測功能，再測效能
- 先測常見情況，再測異常情況
- 先測影響層面大的問題，再測影響層面小的問題
- 先測必須測試的部分(客戶要求)，再測可選擇或未要求測試的部分
- 先測變更過的部分，再測沒有變更的部分(尤其在版本變更的情況下)

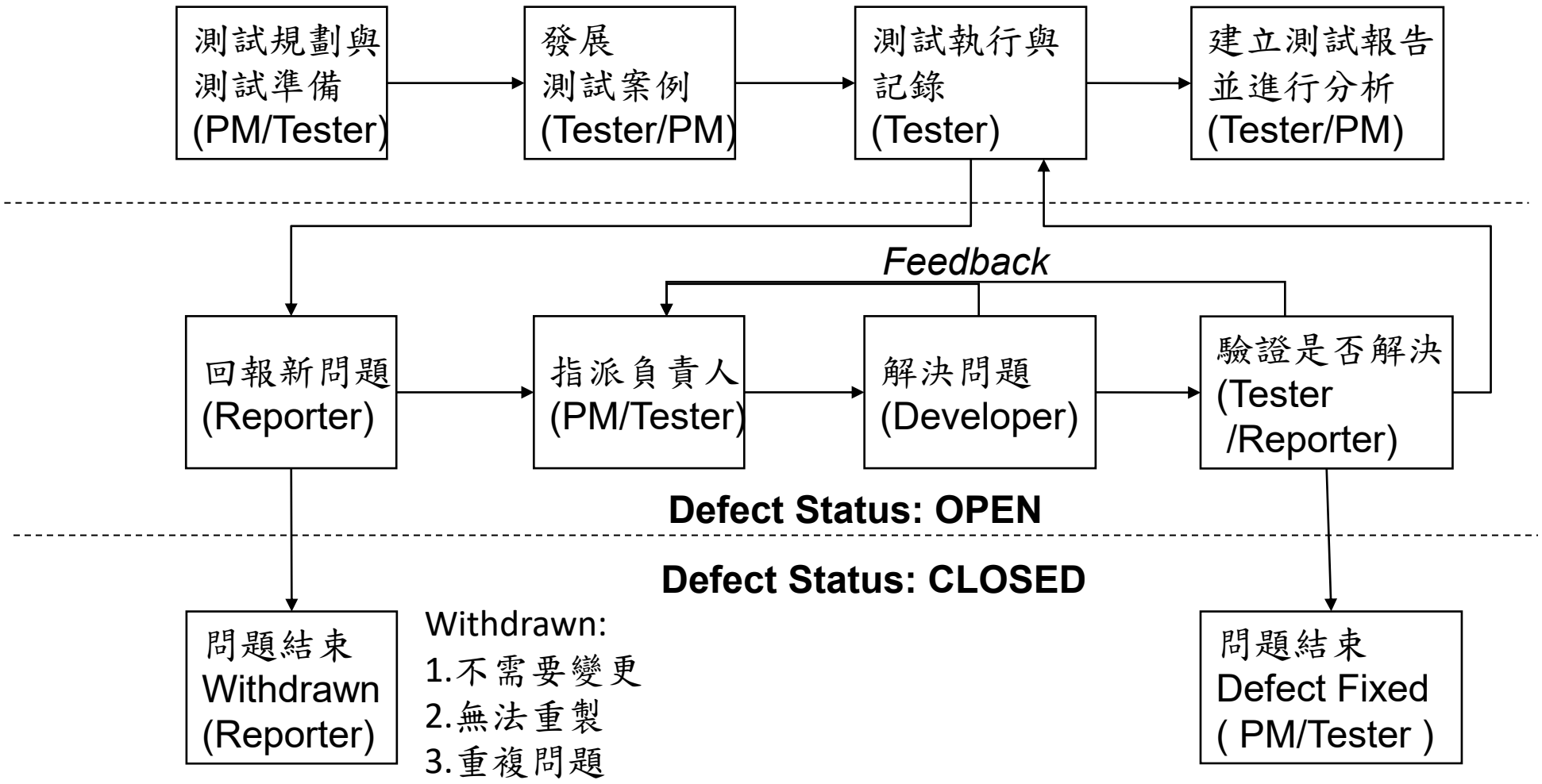
執行軟體測試的指導原則

- 測試環境應獨立於發展環境，以免團隊間的互相干擾，並可完整地確認與模擬軟體執行環境
- 隨時彙總測試結果、統計測試進度及產出測試資料
- 定期召開測試工作會議，進行測試經驗的記錄與交流，並提出可能的改善建議
- 若有程式模組修正缺失，應執行迴歸測試(Regression Testing)的工作，用以確認修正舊問題後不會再次引發新問題
  - Q: 如何規劃迴歸測試案例？

Defect (Bug) 處理流程

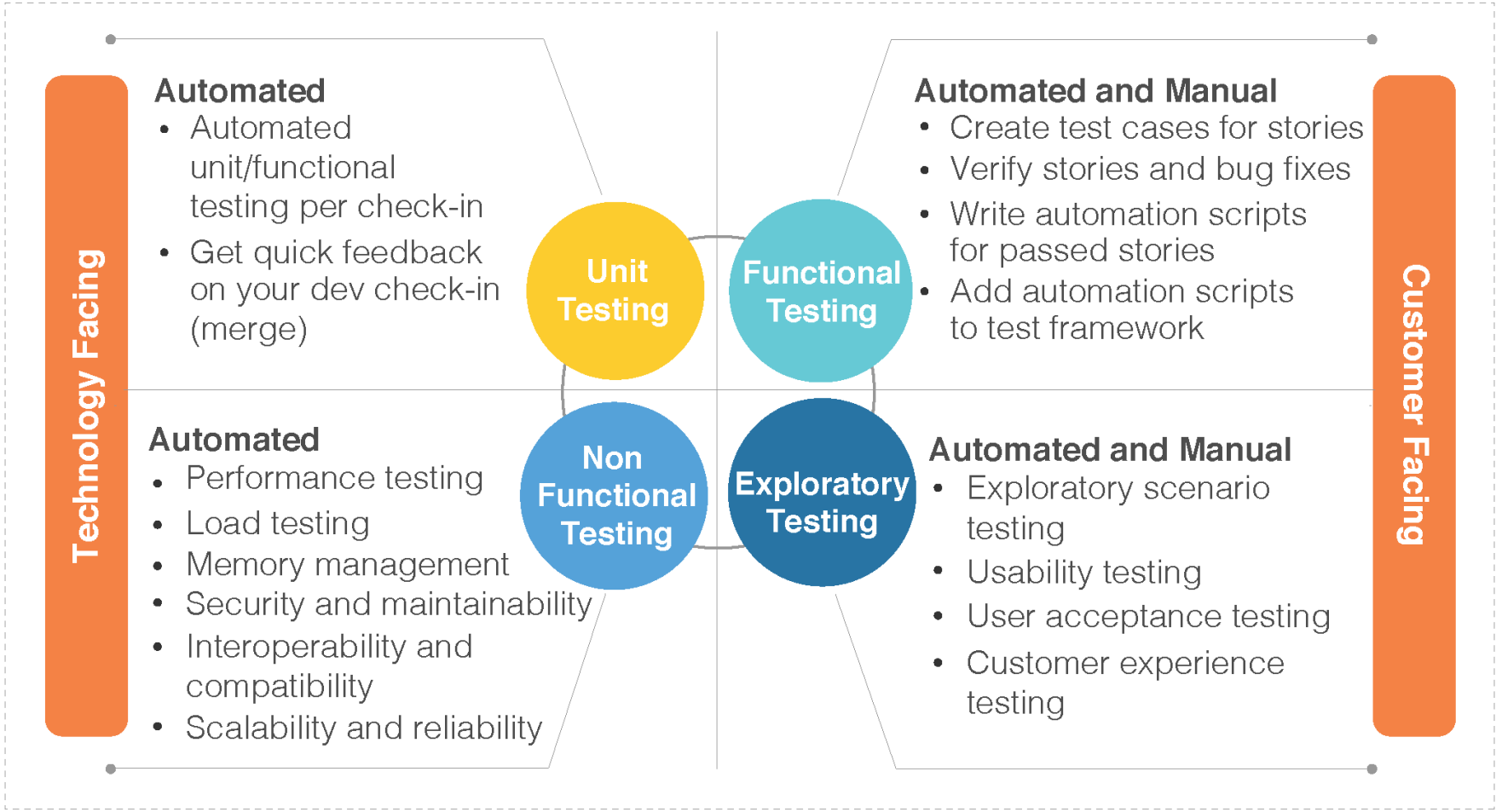


測試與Defect處理流程



自動化測試

自動化測試與手動測試



https://www.mobilelive.ca/blog/why-test-automation

手動與自動化測試比較1

- 手動方式進行測試
  - 優點：
    - 容易模擬使用者行為
  - 缺點：
    - 成本昂貴、耗時
    - 容易出錯，需人工紀錄執行過程與結果

手動與自動化測試比較2

- 自動化方式進行測試(撰寫測試程式或錄製測試腳本)
  - 優點：
    - 可以降低測試時間與成本
    - 自動記錄執行過程、輔助測試結果統計與分析
    - 可模擬多人同時操作軟體
    - 可自動化測試的執行、測試管理與部署
  - 缺點：
    - 測試程式的撰寫耗時(若採錄製則可降低耗時)
    - 若系統介面持續變化，自動化測試案例可能需重建

自動化功能測試工具: Selenium

- https://www.selenium.dev/
- A portable software-testing framework for web applications.
- Selenium IDE: provide a **record/playback (錄製播放)** tool for authoring tests without the need to learn a test scripting language.
- Selenium WebDriver: accepts commands (sent in Selenese, or via a Client API) and sends them to a browser.
  - Fully test automation

自動化功能測試工具: Cypress

- https://www.cypress.io/how-it-works
- 以 JavaScript 撰寫測試程式，可直接在瀏覽器中執行，適合現代前端開發流程。
- 提供即時重新執行(real-time reloading)、自動等待(automatic waiting)、螢幕截圖、 Time Travel等功能，方便觀察每一步測試行為。
- 常用於 Web 應用程式的端對端與整合測試，可搭配 CI/CD pipeline 自動化執行。

自動化功能測試工具: Playwright

- https://playwright.dev/
- Playwright是微軟開發的現代 E2E 測試框架。
- 支援 Chromium、Firefox、WebKit 等多種瀏覽器，可在 Windows、Linux、macOS 上執行。
- 支援多種程式語言(TypeScript/JavaScript、Python、Java、.NET)，提供自動等待、平行執行、多瀏覽器情境等功能。
- 適合需要跨瀏覽器、跨平台保證行為一致的 Web 應用程式端對端測試。

自動化功能測試工具: Rapi/SideeX

- https://chromewebstore.google.com/detail/rapi-recorder/ajondaiaeoipdkpjeoljbjimmfenohin?pli=1
- Rapi is a free and open source smart record-playback automated web application testing tool.
- Rapi presents as an extended version of Selenium IDE with a number of superior expanded automation capabilities.
- Demo: https://www.youtube.com/watch?v=zgZoiPchZpk

Any Question?

